C++ Final Projects 2026

Language	C++ (external libraries may be used when needed)
Suggested completion time	1 month and 2 weeks

General Requirements

- Each project must be implemented in C++.
- External libraries may be used when justified and properly documented.
- The final submission should include the complete source code and short documentation in PDF format.
- The documentation should contain: project description, compilation instructions, usage instructions, screenshots of results, and graphs where appropriate.
- If a project is completed by multiple students, the documentation must begin with a table specifying which part of the project was completed by which student.

Project Overview

No.	Project Title	Team Size
1	Big Data Analysis	1 student
2	Text Pattern Matching	1 student
3	Data Compression and Decompression	1 student
4	Statistical Analysis Tool	1 student
5	Traveling Salesman Problem Solver with Genetic Algorithm	1-2 students
6	Network Protocol Simulator with TCP/IP and UDP	1-2 students
7	RSA Cryptographic Algorithm Implementation	1 student
8	Simple Database Management System with ORM	1-2 students
9	Natural Language Processing Tool	1 student
10	Discrete Fourier Transform (DFT) Calculator	1 student
11	String Manipulation Program with File Handling	1 student
12	Numerical Integration Tool	1 student
13	Root Finding Algorithm Implementation	1 student

No.	Project Title	Team Size
14	Linear Regression Analysis Tool	1 student
15	Maze Solver and Pathfinding Visualizer	1 student
16	Image Processing Tool with Basic Filters	1 student
17	CPU Scheduling Simulator	1 student
18	Cache Memory Simulator	1 student
19	Page Replacement Algorithm Simulator	1 student
20	Sudoku Solver and Generator	1 student
21	Graph Analysis Tool	1 student
22	Recommendation System Based on User Ratings	1 student

Detailed Project Descriptions

1. Big Data Analysis (1 student)

- Implement a correlation algorithm for analyzing large datasets.
- Correlation is a statistical measure determining the degree of relationship between two sets of data. One popular method for calculating correlation is Pearson correlation, which measures the linear relationship between two variables.
- Implement the Pearson correlation coefficient for numerical datasets.
- Allow users to load data from a text or CSV file and select the variables to be compared.
- Test the program on datasets of different sizes and analyze execution time and memory usage.
- Present the results in a clear form, including the computed correlation coefficient and interpretation of the result.
- Include error handling for missing values, invalid file format, and inconsistent data lengths.

2. Text Pattern Matching (1 student)

- Develop a text pattern matching program in C++ to find occurrences of a pattern within a given text.
- Choose a straightforward algorithm such as the naive algorithm, Knuth-Morris-Pratt (KMP) algorithm, or Boyer-Moore algorithm for pattern matching.
- Design a user-friendly interface allowing users to input a text and a pattern to search for.
- Implement the selected pattern matching algorithm efficiently to handle both short and long texts.
- Test the program with various datasets containing both short and long texts to verify its correctness and efficiency.
- Provide clear output indicating the positions or indices of the pattern occurrences in the input text.
- Include error handling mechanisms to handle cases where the pattern is not found in the text or invalid input is provided.

3. Data Compression and Decompression (1 student)

- Design and implement a data compression algorithm (e.g., Huffman coding or Lempel-Ziv-Welch) using C++.
- Create a program that compresses files efficiently while maintaining data integrity.
- Develop a corresponding decompression algorithm to restore compressed data to its original form.
- Measure the compression ratio and execution time for different types of input data.
- Verify correctness by comparing the original and decompressed files.
- Include error handling for corrupted compressed files and unsupported input formats.

4. Statistical Analysis Tool (1 student)

- Create a user-friendly statistical analysis tool in C++.
- Allow users to input data and choose from a set of basic statistical analyses.
- Implement functions for common statistical operations such as mean, median, mode, and standard deviation.
- Implement algorithms for hypothesis testing and confidence interval estimation.
- Include simple visualization options such as histograms or scatter plots.
- Ensure the tool can handle different types and sizes of datasets.
- Test the tool with various datasets to ensure accuracy and reliability.

5. Traveling Salesman Problem Solver with Genetic Algorithm (1-2 students)

- Develop a program in C++ to solve the Traveling Salesman Problem (TSP) using a genetic algorithm (GA).
- Represent cities as nodes and distances between them as edges in a graph.
- Define suitable representations for individuals (tours) in the genetic algorithm, for example permutations of cities.
- Implement genetic operators such as selection, crossover, and mutation to evolve solutions over generations.
- Use a fitness function to evaluate the quality of tours based on total tour length.
- Test the solver on benchmark instances with varying numbers of cities to assess scalability and efficiency.
- Analyze the impact of parameters such as population size, mutation rate, and crossover strategy on the algorithm's performance.

6. Network Protocol Simulator with TCP/IP and UDP (1-2 students)

- Develop a network protocol simulator in C++ to emulate communication between network nodes, with emphasis on TCP and UDP.
- Design modules to represent network nodes, communication channels, and protocol stack layers.
- Implement simplified TCP and UDP mechanisms, including connection establishment, reliable transfer, and datagram delivery.
- Incorporate packet loss, delay, and corruption to simulate realistic network conditions.
- Develop mechanisms for logging network events and capturing packet traces.
- Measure performance metrics such as throughput, latency, and packet loss under different settings.
- Document the assumptions and simplifications made in the simulator design.

7. RSA Cryptographic Algorithm Implementation (1 student)

- Implement the RSA cryptographic algorithm in C++ for secure data encryption and decryption.
- Develop functions for generating RSA key pairs using prime number generation, modular arithmetic, and number theory concepts.
- Implement functions for encrypting and decrypting text or binary data using the generated key pairs.
- Provide options for configuring key sizes such as 1024-bit, 2048-bit, and 4096-bit.
- Test the implementation with various input data types and sizes to verify correctness and performance.
- Analyze the trade-off between security level and execution time.

8. Simple Database Management System with ORM (1-2 students)

- Create a basic database management system (DBMS) in C++.
- Design a simple data model for storing and retrieving information.
- Build modules for defining schemas, adding, updating, deleting data, and running basic queries.
- Implement a basic query language interface allowing users to interact with the DBMS.
- Include support for indexing to speed up data retrieval.
- Integrate Object-Relational Mapping (ORM) functionality enabling mapping of database entities to C++ classes.
- Test the DBMS with sample data to ensure it functions as expected.

9. Natural Language Processing Tool (1 student)

- Develop a basic natural language processing (NLP) tool in C++ for analyzing English text.
- Create functions for common text processing tasks such as tokenization, part-of-speech tagging, and named entity recognition.
- Use existing libraries or simple algorithms where appropriate.
- Provide a user-friendly interface for inputting text and displaying the results of the analysis.
- Test the tool with various text samples to ensure accuracy and reliability.
- Discuss the limitations of the chosen approach.

10. Discrete Fourier Transform (DFT) Calculator (1 student)

- Develop a program to calculate the Discrete Fourier Transform (DFT) of a given sequence of discrete data points.
- Implement the DFT algorithm using complex arithmetic and trigonometric functions.
- Provide options for users to input the sequence of data points and specify the sampling frequency.
- Display the magnitude and phase spectra of the DFT coefficients.
- Validate correctness by comparing the results with known analytical solutions or by applying the inverse DFT.
- Include example test signals and discuss the observed frequency content.

11. String Manipulation Program with File Handling (1 student)

- Develop a program in C++ to manipulate strings and perform file handling operations.
- Design a user-friendly interface allowing users to choose from several string manipulation operations.
- Implement operations such as reversing a string, counting characters, searching substrings, or changing letter case.

- Use classes or structures to organize related data and operations.
- Integrate file handling capabilities to read input from a text file and write output to another file.
- Provide options for users to specify input and output file paths.
- Ensure robust error handling for file-related errors and unexpected input.

12. Numerical Integration Tool (1 student)

- Create a numerical integration program in C++ to approximate definite integrals of functions using methods such as the trapezoidal rule or Simpson's rule.
- Implement functions for evaluating integrals of user-defined functions over specified intervals.
- Provide options for users to input the function, the integration interval, and the desired precision.
- Use loops and numerical methods to compute the approximation and output the result.
- Validate accuracy by comparing results with known analytical solutions for test functions.
- Discuss the effect of step size or number of intervals on the result.

13. Root Finding Algorithm Implementation (1 student)

- Implement a root-finding algorithm such as Newton's method or the bisection method in C++ to find approximate solutions to equations.
- Design functions to encapsulate the root-finding logic, taking into account convergence criteria and error tolerance.
- Allow users to input the equation to be solved and the initial guess or interval.
- Test the algorithm with various equations, including polynomial and transcendental functions.
- Evaluate convergence properties and accuracy for different examples.
- Include appropriate validation and error reporting for invalid input.

14. Linear Regression Analysis Tool (1 student)

- Create a program in C++ to perform linear regression analysis on a set of data points.
- Implement algorithms for calculating least-squares regression coefficients and goodness-of-fit metrics.
- Allow users to input datasets consisting of pairs of independent and dependent variables.
- Display the regression equation, coefficients, and a graphical representation of the fitted line together with the original data points.
- Include options for additional analyses such as weighted regression or polynomial regression.
- Test the program on several datasets and discuss the quality of the fit.

15. Maze Solver and Pathfinding Visualizer (1 student)

- Develop a C++ application for generating and solving mazes.
- Implement one maze generation algorithm and at least one pathfinding algorithm such as BFS, DFS, Dijkstra's algorithm, or A*.
- Allow users to generate mazes of different sizes and visualize the discovered path.
- Display statistics such as path length, number of visited nodes, and execution time.
- Test the program on mazes of varying sizes and compare the implemented algorithms.
- Include a short discussion of algorithmic complexity and observed performance.

16. Image Processing Tool with Basic Filters (1 student)

- Create a C++ program for basic image processing operations.
- Allow users to load an image and apply selected filters such as grayscale conversion, thresholding, blur, edge detection, and contrast adjustment.
- Use appropriate data structures to represent images and process pixel values efficiently.
- Provide options for saving the processed image to a file.
- Include performance tests for different image sizes and compare the computational cost of the implemented operations.
- Document the supported image formats and any external libraries used.

17. CPU Scheduling Simulator (1 student)

- Develop a C++ simulator for selected CPU scheduling algorithms.
- Implement algorithms such as First-Come First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin.
- Allow users to define processes with arrival times, burst times, and priorities.
- Display scheduling results including waiting time, turnaround time, and a Gantt chart representation.
- Compare the performance of the algorithms on different process sets and discuss their advantages and disadvantages.
- Provide clear documentation of the scheduling assumptions.

18. Cache Memory Simulator (1 student)

- Create a C++ simulator of cache memory behavior.
- Implement selected cache mapping strategies such as direct-mapped, fully associative, and set-associative cache.
- Simulate read and write operations for a sequence of memory addresses.
- Measure and display cache statistics such as hit rate, miss rate, and average memory access time.
- Test the simulator using different cache sizes, block sizes, and replacement policies.
- Discuss how configuration parameters affect performance.

19. Page Replacement Algorithm Simulator (1 student)

- Develop a C++ program that simulates virtual memory page replacement algorithms.
- Implement algorithms such as FIFO, LRU, and Optimal page replacement.
- Allow users to input page reference strings and specify the number of available frames.
- Display the sequence of memory states and compute the number of page faults for each algorithm.
- Compare the results for different workloads and discuss the behavior of each method.
- Include examples that highlight major differences between the algorithms.

20. Sudoku Solver and Generator (1 student)

- Create a C++ application for solving and generating Sudoku puzzles.
- Implement a solving algorithm based on backtracking or constraint propagation.
- Add functionality for generating valid Sudoku boards with different difficulty levels.
- Provide a simple interface for entering puzzles manually or loading them from a file.
- Measure the performance of the solver on puzzles of varying complexity and present the results.

- Explain how correctness of generated puzzles is ensured.

21. Graph Analysis Tool (1 student)

- Develop a C++ tool for graph creation and analysis.
- Allow users to input directed or undirected graphs and perform selected operations such as BFS, DFS, shortest path search, connected components detection, and cycle detection.
- Display the graph structure and analysis results in a clear form.
- Provide tests for graphs of different sizes and densities.
- Compare the computational cost of selected graph operations.
- Document the graph representation used in the implementation.

22. Recommendation System Based on User Ratings (1 student)

- Create a simple recommendation system in C++ based on user-item ratings.
- Implement one recommendation method such as user-based collaborative filtering, item-based collaborative filtering, or similarity-based recommendation using cosine similarity or Pearson correlation.
- Allow users to load ratings data from a file and generate recommendations for selected users.
- Display similarity values, predicted ratings, and top recommended items.
- Test the system on a sample dataset and evaluate recommendation quality using simple error measures.
- Discuss the limitations of the chosen recommendation method.